

澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU



信息學院
Faculdade de Ciência da Informação e Computação
Faculty of Information Science and Computing

Autoencoders

EMEN 7029 & EMEN 7104 - Special Topic In Electromechanical Engineering / Robotics

Qingbiao Li

University of Macau

Review 2
Compression 20
Autoencoders 27
Convolutional Autoencoders 41
Recurrent Autoencoders 51
Applications and Objectives 58
Emergent Intelligence 66

Review

Review

Convolution works over inputs of any variables (time, space, etc)

Recurrent neural networks only work with time

Convolution uses locality and translation equivariance properties

Recurrent models do not assume locality or equivariance

Choose convolution or RNN based on problem structure

Review

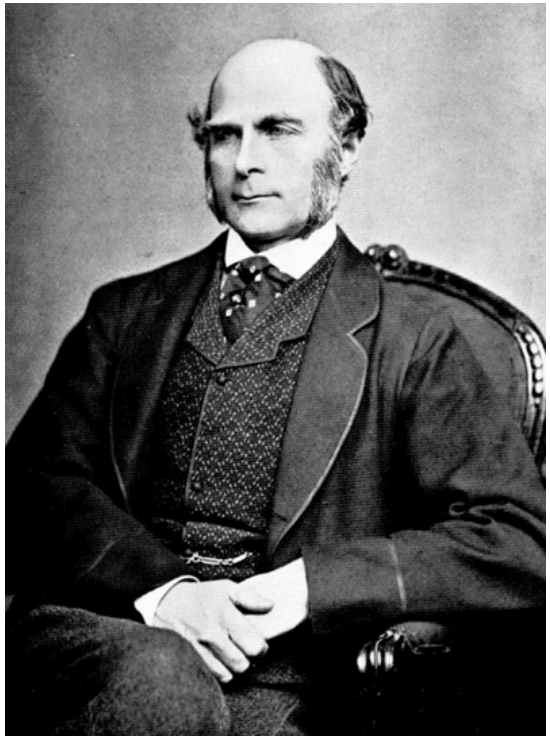
How do humans process temporal data?

Humans only perceive the present

Humans process temporal data by storing and recalling memories

Review

Francis Galton (1822-1911) photo composite memory



Composite photo of members of a party



Review

Task: Find a mathematical model of how our mind represents memories

$$X : \mathbb{R}^{h \times w} \quad \text{People you see at the party}$$

$$H : \mathbb{R}^{h \times w} \quad \text{The image in your mind}$$

$$f : X^T \times \Theta \mapsto H$$

Composite photography/memory uses a weighted sum

$$f(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{T} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_i$$

Review

$$f(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{T} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_i$$

What if we see a new face? $$f(\boldsymbol{X}, \boldsymbol{\theta}) = \left(\sum_{i=1}^{T} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_i \right) + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T+1}$$

We repeat the same process for each new face

We can rewrite f as a **recurrent function**

Review

Rewrote composite memory as a recurrent function

$$f(\boldsymbol{X}, \boldsymbol{\theta}) = \underbrace{\left(\sum_{i=1}^{T} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_i \right)}_{\boldsymbol{h}} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T+1}$$

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

Review

$$\boldsymbol{x} \in \mathbb{R}^{d_x}, \quad \boldsymbol{h} \in \mathbb{R}^{d_h} \qquad\qquad f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

$$\boldsymbol{h}_1 = f(\mathbf{0}, \boldsymbol{x}_1, \boldsymbol{\theta}) = \mathbf{0} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_1$$

$$\boldsymbol{h}_2 = f(\boldsymbol{h}_1, \boldsymbol{x}_2, \boldsymbol{\theta}) = \boldsymbol{h}_1 + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_2$$

$$\boldsymbol{h}_3 = f(\boldsymbol{h}_2, \boldsymbol{x}_3, \boldsymbol{\theta}) = \boldsymbol{h}_2 + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_3$$

$$\vdots$$

$$\boldsymbol{h}_T = f(\boldsymbol{h}_{T-1}, \boldsymbol{x}_T, \boldsymbol{\theta}) = \boldsymbol{h}_{T-1} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$

Review

This model remembers everything

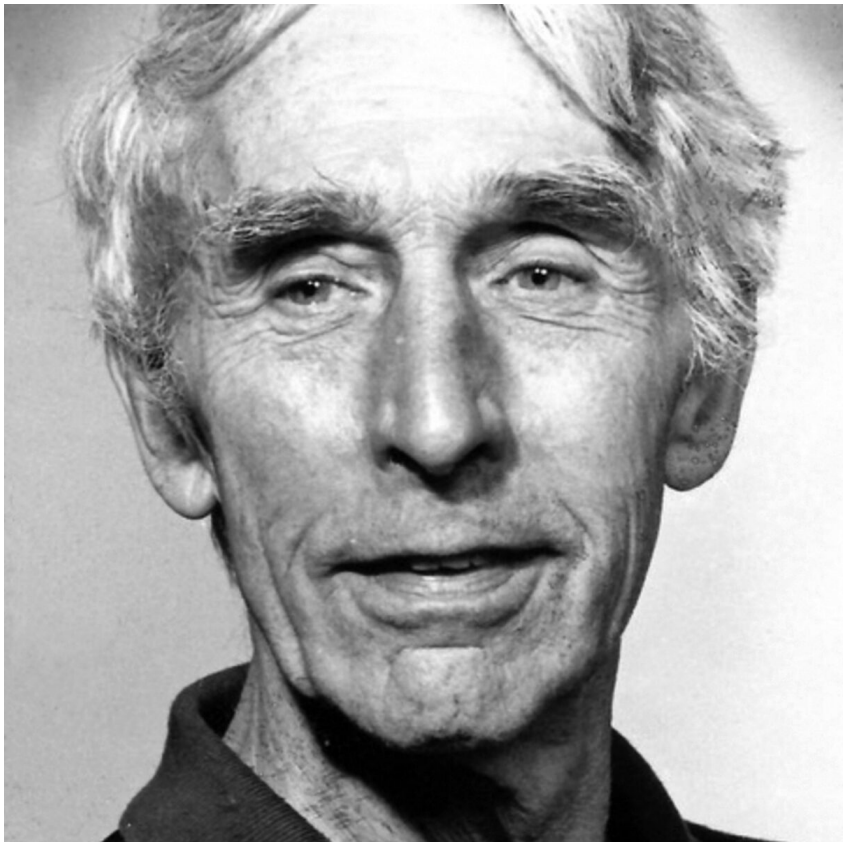
But $\boldsymbol{h}$ is a fixed size, what if T is very large?

If we keep adding and adding $\boldsymbol{x}$ into $\boldsymbol{h}$, we will run out of space

Humans forget old information

Review

Murdock (1982)



$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \gamma \boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}; \quad 0 < \gamma < 1$$

Key Idea: $\lim_{T \to \infty} \gamma^T = 0$

Let $\gamma = 0.9$

$$0.9 \cdot 0.9 \cdot \boldsymbol{h} = 0.81\boldsymbol{h}$$

$$0.9 \cdot 0.9 \cdot 0.9 \cdot \boldsymbol{h} = 0.72\boldsymbol{h}$$

$$0.9 \cdot 0.9 \cdot 0.9 \cdot \ldots \cdot \boldsymbol{h} = 0$$

Review

$$\boldsymbol{h}_T = \gamma^3 \boldsymbol{h}_{T-3} + \gamma^2 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-2} + \gamma \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-1} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$

$$\boldsymbol{h}_T = \gamma^T \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_1 + \gamma^{T-1} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_2 + \ldots + \gamma^2 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-2} + \gamma \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-1} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$

Review

Our function f is just defined for a single X

$$f : H \times X \times \Theta \mapsto H$$

To extend f to sequences, we scan f over the inputs

$$\text{scan}(f) : H \times X^T \times \Theta \mapsto H^T$$

$$\text{scan}(f)\left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}\right) = \begin{bmatrix} \boldsymbol{h}_1 \\ \boldsymbol{h}_2 \\ \vdots \\ \boldsymbol{h}_T \end{bmatrix}$$

Review

$$f : H \times X \times \Theta \mapsto H, \quad \text{scan}(f) : H \times X^T \times \Theta \mapsto H^T$$

$$\text{scan}(f)\left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}\right) = \begin{bmatrix} \boldsymbol{h}_1 \\ \boldsymbol{h}_2 \\ \vdots \\ \boldsymbol{h}_T \end{bmatrix}$$

$$\text{scan}(f)\left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}\right) = \begin{bmatrix} f(\boldsymbol{h}_0, \boldsymbol{x}_1, \boldsymbol{\theta}) \\ f(\boldsymbol{h}_1, \boldsymbol{x}_2, \boldsymbol{\theta}) \\ \vdots \\ f(\boldsymbol{h}_{T-1}, \boldsymbol{x}_T, \boldsymbol{\theta}) \end{bmatrix} = \begin{bmatrix} f(\boldsymbol{h}_0, x_1) \\ f(f(\boldsymbol{h}_0, \boldsymbol{x}_1), \boldsymbol{x}_2) \\ \vdots \\ f(...f(\boldsymbol{h}_0, \boldsymbol{x}_1)..., \boldsymbol{x}_T) \end{bmatrix}$$

Review

Let g define the memory recall function

$$g : H \times X \times \Theta \mapsto Y$$

g searches your memories h using the input x, to produce output y

$\boldsymbol{x}$: "What is your favorite ice cream flavor?"

$\boldsymbol{h}$: Everything you remember (hometown, birthday, etc)

g : Searches your memories for ice cream information, and responds "chocolate"

Review

Step 1: Perform scan to find recurrent states

$$\begin{bmatrix} \boldsymbol{h}_1 \\ \vdots \\ \boldsymbol{h}_T \end{bmatrix} = \text{scan}(f) \left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}_f \right)$$

Step 2: Perform recall on recurrent states

$$\begin{bmatrix} \hat{\boldsymbol{y}}_1 \\ \vdots \\ \hat{\boldsymbol{y}}_T \end{bmatrix} = \begin{bmatrix} g(\boldsymbol{h}_1, \boldsymbol{x}_1, \boldsymbol{\theta}_g) \\ \vdots \\ g(\boldsymbol{h}_T, \boldsymbol{x}_T, \boldsymbol{\theta}_g) \end{bmatrix}$$

Review

The simplest recurrent neural network is the **Elman Network**

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}_1^\top \boldsymbol{h} + \boldsymbol{\theta}_2^\top \overline{\boldsymbol{x}})$$

$$g(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \boldsymbol{\theta}_3^\top \boldsymbol{h}$$

Review

Add forgetting

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \sigma\left(\boldsymbol{\theta}_1^\top \boldsymbol{h} \odot f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) + \boldsymbol{\theta}_2^\top \overline{\boldsymbol{x}}\right)$$

$$f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \sigma\left(\boldsymbol{\theta}_1^\top \overline{\boldsymbol{x}} + \boldsymbol{\theta}_2^\top \boldsymbol{h}\right)$$

When $f_{\text{forget}} < 1$, we forget some of our memories!

Through gradient descent, neural network learns which memories to forget

Review

Minimal gated unit (MGU) is a modern RNN

MGU defines two helper functions

$$f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \sigma\left(\boldsymbol{\theta}_1^\top \overline{\boldsymbol{x}} + \boldsymbol{\theta}_2^\top \boldsymbol{h}\right)$$

$$f_2(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \underbrace{\sigma\left(\boldsymbol{\theta}_3^\top \overline{\boldsymbol{x}} + \boldsymbol{\theta}_4^\top f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) \odot \boldsymbol{h}\right)}_{\text{Make new memories}}$$

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \underbrace{f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) \odot \boldsymbol{h}}_{\text{Forget old memories}} + \underbrace{\left(1 - f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta})\right)}_{\text{Replace}} \odot \underbrace{f_2(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta})}_{\text{New memories}}$$

Only forget when we have new information to remember

Compression

Compression



Question: You watch a film. How do you communicate information about the film with a friend?

Answer: Fat panda becomes kung fu master to defend his village from an evil snow leopard.

Question: What is missing?

Answer: Goose father, tortoise teacher, tiger friend, magic scroll, etc

Compression

When you discuss films with friends, you summarize them

This is a form of **compression**

$$f\left(\begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_n \end{bmatrix}\right) = \text{Fat panda saves village}$$

In compression, we reduce the size of data by either:

- Rewriting data in more efficient format (lossless)
- Removing less important data (lossy)

Let us examine a more principled form of video compression

Compression

Kung Fu Panda in 4k UHD

$$X \in \{0, ..., 255\}^{3 \times 3840 \times 2160}, \; X^{90 \times 60 \times 24}$$

Question: 1 byte represents 256 values. How many GB?

Answer: 3000 GB

But if you download films, you know they are smaller than 3 TB

Usually you download an `mp4` or `mpeg4` file

`mp4` uses `H.264` video **encoder** to compress information

Compression

`H.264` encoder selects 16×16 pixel blocks, estimates shift between frames, applies cosine transform, ...

The result is an `.mp4` file Z

$$f : X^t \mapsto Z \qquad\qquad Z \in \{0,1\}^k$$

Question: What is the size of Z in GB?

Answer: 60 GB, original size was 3000 GB

We achieve a compression ratio of $3000 \text{ GB} \ / \ 60 \text{ GB} = 50$

Compression

What happens on your computer when you watch Kung Fu Panda?

Download $\boldsymbol{z} \in Z$ from the internet $\quad Z \in \{0,1\}^n$

Information is no longer pixels, it is a string of bits

We must **decode** $\boldsymbol{z}$ back into pixels

We need to undo (invert) the encoder f

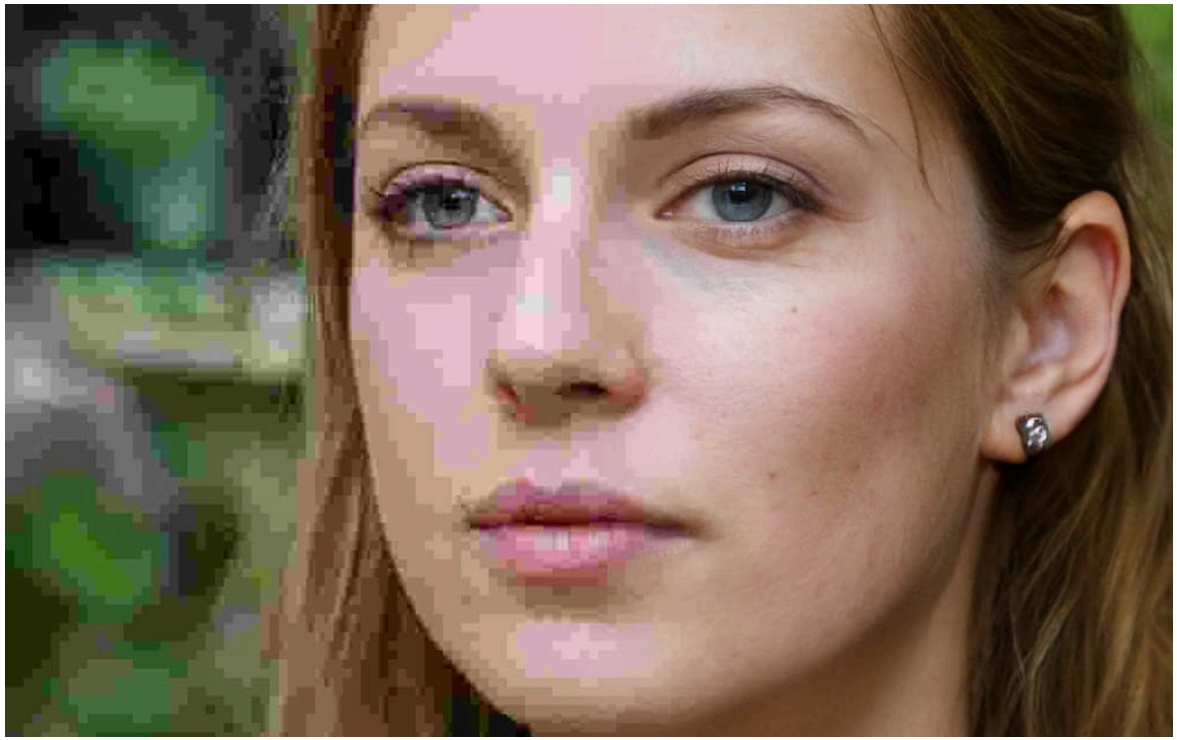
$$f : X^t \mapsto Z$$

$$f^{-1} : Z \mapsto X^t$$

You CPU has an `H.264` hardware decoder to make this fast

Compression

Compression may be **lossy** or **lossless**



Question: Which is `H.264`?

Autoencoders

Autoencoders

Encoders and decoders for images, videos, and music are functions

Neural networks can represent any continuous function

We can use neural networks to represent encoders and decoders

$$f : X \times \Theta \mapsto Z$$

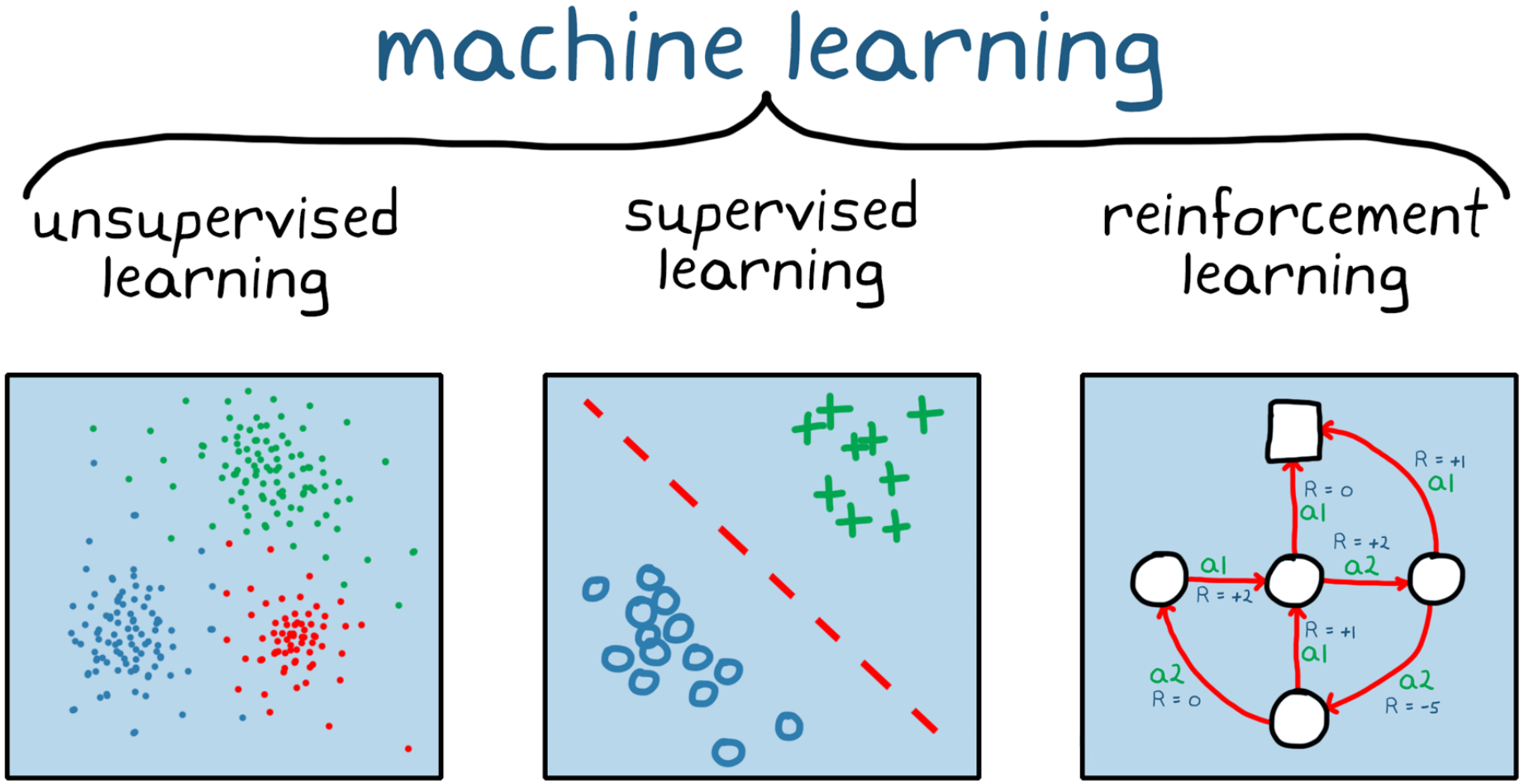
$$f^{-1} : Z \times \Theta \mapsto X$$

We call this an **autoencoder**

Notice there is no Y this time

Training autoencoders is different than what we have seen before

Autoencoders



Autoencoders

In supervised learning, humans provide the model with **inputs** $\boldsymbol{X}$ and corresponding **outputs** $\boldsymbol{Y}$

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_{[1]} & \boldsymbol{x}_{[2]} & \dots & \boldsymbol{x}_{[n]} \end{bmatrix}^\top \quad \boldsymbol{Y} = \begin{bmatrix} \boldsymbol{y}_{[1]} & \boldsymbol{y}_{[2]} & \dots & \boldsymbol{y}_{[n]} \end{bmatrix}^\top$$

In unsupervised learning, humans only provide **input**

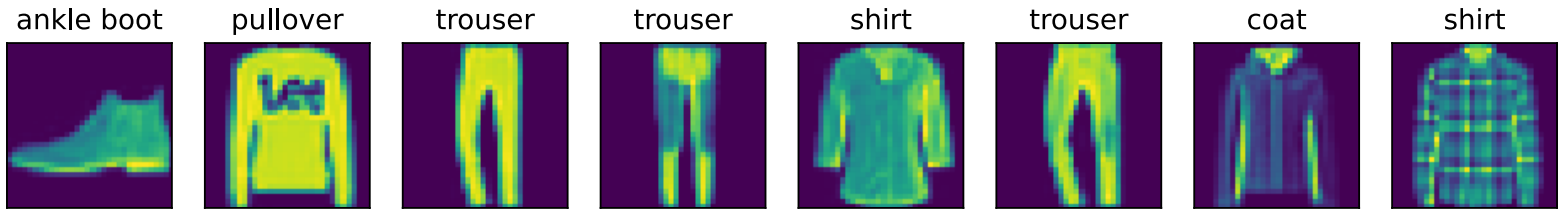
$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_{[1]} & \boldsymbol{x}_{[2]} & \dots & \boldsymbol{x}_{[n]} \end{bmatrix}^\top$$

The training algorithm will learn **unsupervised** (only from $\boldsymbol{X}$)

- Humans do not need to provide labels!

Autoencoders

Task: Compress images for your clothing website to save on costs



$$X \in [0,1]^{d_x}$$

$$Z \in \mathbb{R}^{d_z}$$

$$d_x : 28 \times 28$$

$$d_z : 3$$

$$f : X \times \Theta \mapsto Z$$

$$f^{-1} : Z \times \Theta \mapsto X$$

What is the structure of f, f^{-1}?

How do we find $\boldsymbol{\theta}$?

Autoencoders

Let us find f, then find the inverse f^{-1}

Start with a perceptron

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sigma\left(\boldsymbol{\theta}^\top \overline{\boldsymbol{x}}\right); \quad \boldsymbol{\theta} \in \mathbb{R}^{(d_x+1)\times d_z}$$

$$\boldsymbol{z} = \sigma\left(\boldsymbol{\theta}^\top \overline{\boldsymbol{x}}\right)$$

Solve for $\boldsymbol{x}$ to find the inverse

$$\sigma^{-1}(\boldsymbol{z}) = \sigma^{-1}\left(\sigma\left(\boldsymbol{\theta}^\top \overline{\boldsymbol{x}}\right)\right)$$

$$\sigma^{-1}(\boldsymbol{z}) = \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

Autoencoders

$$\sigma^{-1}(\boldsymbol{z}) = \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

$$\left(\boldsymbol{\theta}^\top\right)^{-1}\sigma^{-1}(\boldsymbol{z}) = \left(\boldsymbol{\theta}^\top\right)^{-1}\boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

$$\left(\boldsymbol{\theta}^\top\right)^{-1}\sigma^{-1}(\boldsymbol{z}) = \boldsymbol{I}\overline{\boldsymbol{x}}$$

$$\left(\boldsymbol{\theta}^\top\right)^{-1}\sigma^{-1}(\boldsymbol{z}) = \overline{\boldsymbol{x}}$$

$$f^{-1}(\boldsymbol{z}, \boldsymbol{\theta}) = \left(\boldsymbol{\theta}^\top\right)^{-1}\sigma^{-1}(\boldsymbol{z})$$

Autoencoders

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \overline{\boldsymbol{x}}) \quad f^{-1}(\boldsymbol{z}, \boldsymbol{\theta}) = (\boldsymbol{\theta}^\top)^{-1} \sigma^{-1}(\boldsymbol{z}) \qquad \boldsymbol{\theta} \in \mathbb{R}^{(d_x+1)\times d_z}$$

Question: Any issues? **Hint:** What if $d_x \neq d_z$?

Answer: Can only invert square matrices, $\boldsymbol{\theta}^\top$ only invertible if $d_z = d_x$

Question: What kind of compression can we achieve if $d_z = d_x$?

Answer: None! We need $d_z < d_x$ for compression

Look for another solution

Autoencoders

Let us try another way

$$\begin{aligned} \boldsymbol{z} = f(\boldsymbol{x}, \boldsymbol{\theta}_e) &= \sigma\left(\boldsymbol{\theta}_e^\top \overline{\boldsymbol{x}}\right) \\ \boldsymbol{x} = f^{-1}(\boldsymbol{z}, \boldsymbol{\theta}_d) &= \sigma\left(\boldsymbol{\theta}_d^\top \overline{\boldsymbol{z}}\right) \end{aligned}$$

What if we plug $\boldsymbol{z}$ into the second equation?

Autoencoders

Let us try another way

$$\boldsymbol{z} = f(\boldsymbol{x}, \boldsymbol{\theta}_e) = \sigma(\boldsymbol{\theta}_e^\top \overline{\boldsymbol{x}})$$

$$\boldsymbol{x} = f^{-1}(\boldsymbol{z}, \boldsymbol{\theta}_d) = \sigma(\boldsymbol{\theta}_d^\top \overline{\boldsymbol{z}})$$

What if we plug $\boldsymbol{z}$ into the second equation?

$$\boldsymbol{x} = f^{-1}(f(\boldsymbol{x}, \boldsymbol{\theta}_e), \boldsymbol{\theta}_d) = \sigma(\boldsymbol{\theta}_d^\top \boldsymbol{\sigma}(\boldsymbol{\theta}_e^\top \overline{\boldsymbol{x}}))$$

Autoencoders

$$\boldsymbol{x} = f^{-1}(f(\boldsymbol{x}, \boldsymbol{\theta}_e), \boldsymbol{\theta}_d) = \sigma\big(\boldsymbol{\theta}_d^\top \sigma(\boldsymbol{\theta}_e^\top \overline{\boldsymbol{x}})\big)$$

More generally, f, f^{-1} may be any neural network

$$\boldsymbol{x} = f^{-1}(f(\boldsymbol{x}, \boldsymbol{\theta}_e), \boldsymbol{\theta}_d)$$

Turn this into a loss function using the square error

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\theta}) = \sum_{j=1}^{d_x} \Big(x_j - f^{-1}(f(\boldsymbol{x}, \boldsymbol{\theta}_e), \boldsymbol{\theta}_d)_j \Big)^2$$

Forces the networks to compress and reconstruct $\boldsymbol{x}$

Autoencoders

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\theta}) = \sum_{j=1}^{d_x} \left(x_j - f^{-1}(f(\boldsymbol{x}, \boldsymbol{\theta}_e), \boldsymbol{\theta}_d)_j \right)^2$$

Define over the entire dataset

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e\right), \boldsymbol{\theta}_d\right)_j \right)^2$$

We call this the **reconstruction loss**

Autoencoders

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$$

The reconstruction loss is an **unsupervised loss**

- No traditional labels Y!
- Learns from unlabeled data X

Autoencoders

So far, we consider only a perceptron and mean square error

There are many types of autoencoder architectures and objectives

Convolutional Autoencoders

Convolutional Autoencoders





Task: Compress an image

$$1\text{MB} \rightarrow 10\text{kb}$$

For Fashion MNIST we used perceptron

Question: Do we have better networks for images?

Question: Use convolutional network. Why?

Answer: More efficient than perceptrons (data/parameters)

Convolutional Autoencoders

With autoencoders we must contract then expand

$$f : \mathbb{R}^{d_x} \times \Theta \mapsto \mathbb{R}^{d_z} \quad d_z < d_x$$

$$f^{-1} : \mathbb{R}^{d_z} \times \Theta \mapsto \mathbb{R}^{d_x}$$

Question: Do convolution and pooling contract or expand?

Answer: Usually contract, reduce the size of the signal

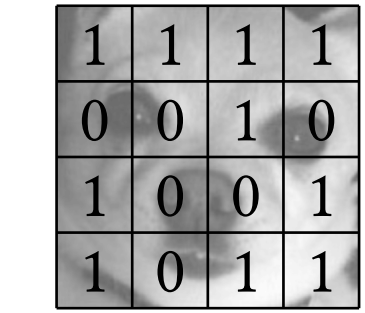
To create a convolutional decoder, we must expand the signal

To expand the signal, we must "invert" convolution and pooling

Convolutional Autoencoders

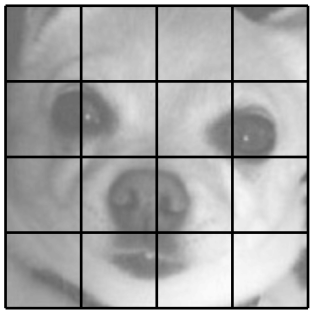
Let us first try and "invert" pooling

Consider mean pooling (adaptive 2×2)



$$\sum$$

$$\frac{1}{4} \cdot \begin{bmatrix} & \\ & \end{bmatrix}$$



$$\sum$$

$$\frac{1}{4} \cdot \begin{bmatrix} 2 & 3 \\ 2 & 3 \end{bmatrix}$$

This "inverse" pooling has two names: **unpooling** and **upsampling**

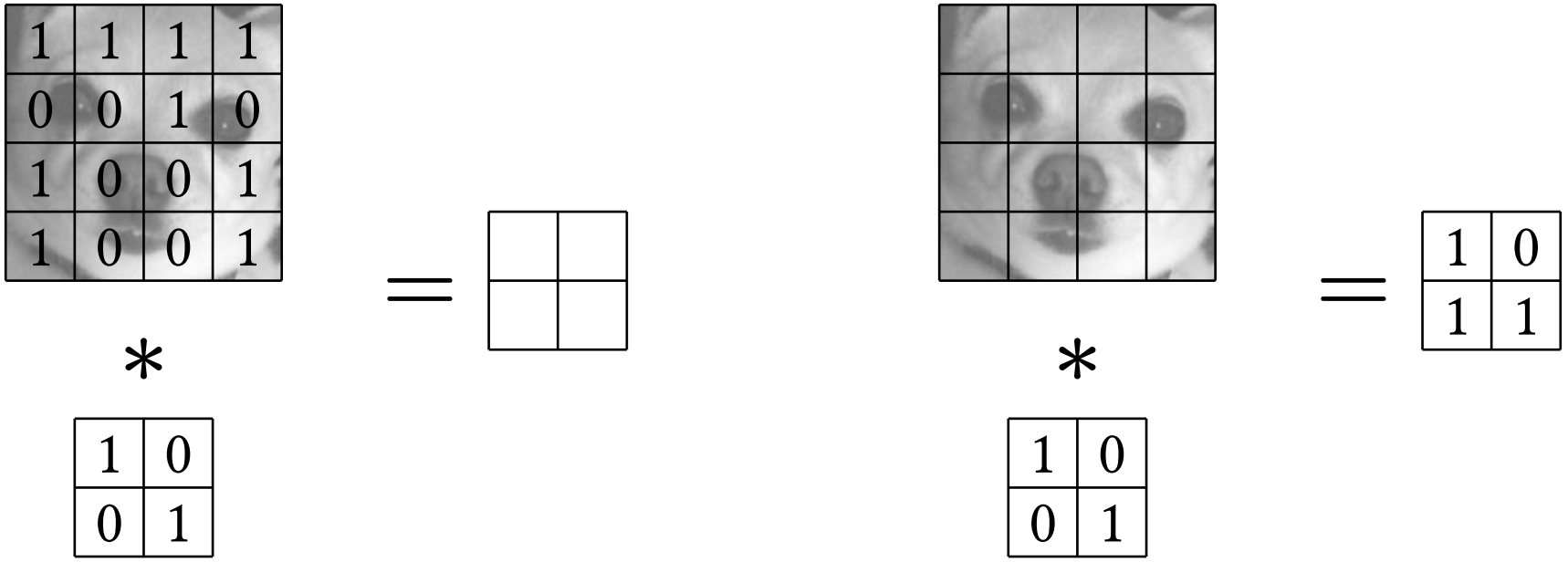
Convolutional Autoencoders

Upsampling does not use parameters!

- Only upscaling z will not provide a good reconstruction
- Need to introduce learned parameters
  - Combine with convolution layers or “inverse” convolution layers

Convolutional Autoencoders

Let us try and “invert” convolution



We call this **deconvolution** or **transposed convolution**

Convolutional Autoencoders

Transposed convolution is similar to upsampling

- Unlike upsampling, we learn the filters (parameters)

New “inverse” operations let us construct a convolutional decoder

- Many different ways to do this

Convolutional Autoencoders

Symmetric convolution-only autoencoder

```
encoder = Sequential(

  Conv2d(3, 16, 3)
  LeakyReLU()

  Conv2d(16, 32, 3)
  LeakyReLU()
  ...
  Conv2d(32, 32, 3)
  Flatten()
)
```

```
decoder = Sequential(
  Unflatten()
  ConvTranspose2d(32, 32, 3)
  LeakyReLU()

  ConvTranspose2d(32, 16, 3)
  LeakyReLU()
  ...
  ConvTranspose2d(16, 3, 3)

)
```

Convolutional Autoencoders

Other architectures use asymmetric `conv -> upsample` blocks

```
encoder = Sequential(

  # Block
  Conv2d()
  MaxPool()
  LeakyReLU()

  ...
  Conv2d()
  Flatten()
)
```

```
decoder = Sequential(
  Unflatten()
  # Block
  Upsample()
  Conv2d()
  LeakyReLU()

  ...
  Conv2d()
)
```

Convolutional Autoencoders

I find that asymmetric upsample autoencoders tend to work better

- `Upsample -> Conv -> Activation`
- Finding filter size, stride, etc that produce the correct shape is painful

I use symmetric autoencoders because they are easier to implement

- `ConvTranspose -> Activation`
- Usually just reorder encoder arguments
  - `Conv2d(A, B, C) -> ConvTranspose2d(B, A, C)`

Recurrent Autoencoders

Recurrent Autoencoders





$\vdots$

Task: Compress entire film (sequence of images)

Step 1: Compress image with CNN $\rightarrow \boldsymbol{x}$

Step 2: Compress $\boldsymbol{x}_1, \ldots \boldsymbol{x}_T \rightarrow \boldsymbol{h}$

Question: What models process a sequence of inputs?

Answer: Recurrent neural networks!

Recurrent Autoencoders

Recall the recurrent neural network type signatures

$$f : H \times X \times \Theta \mapsto H$$

$$\text{scan}(f) : H \times X^T \times \Theta \mapsto H^T$$

Question: This is the encoder, what about decoder?

$$g : H \times X \times \Theta \mapsto X$$

Question: Any problems? **Answer:** Need input to reconstruct input

$$f^{-1} : H \times \Theta \mapsto X$$

$$\text{scan}(f^{-1}) : H \times \Theta \mapsto H^T \times X^T$$

Recurrent Autoencoders

$$f\left(\boldsymbol{h}_0, \underbrace{\text{  }}_{\boldsymbol{x}_1}, \boldsymbol{\theta}_e\right) = \boldsymbol{h}_1$$

$$\vdots$$

$$f\left(\boldsymbol{h}_{T-1}, \underbrace{\text{  }}_{\boldsymbol{x}_T}, \boldsymbol{\theta}_e\right) = \boldsymbol{h}_T$$

Recurrent Autoencoders

$$f(\boldsymbol{h}_T, \boldsymbol{\theta}_d) = \hat{\boldsymbol{h}}_{T-1},$$



$$\boldsymbol{x}_T$$

$$\vdots$$

$$f\left(\hat{\boldsymbol{h}}_1, \boldsymbol{\theta}_d\right) = \hat{\boldsymbol{h}}_0,$$



$$\boldsymbol{x}_T$$

Recurrent Autoencoders

```
# Compress images to latent sequence
zs = vmap(cnn_enc)(frames)
h0 = zeros(d_h) # Initial state
# Compress sequence to latent vector
hT, hs = scan(rnn_enc, init=h0, xs=zs)
# Final state should hold all film info
# Decompress final state into latent sequence
recon_zT, recon_zs = scan(rnn_dec, init=hT, length=T)
# Decompress latent sequence into images
recon_frames = vmap(cnn_dec)(recon_zs)
# Compute loss and update params
grads = grad(lambda x, xhat: mean((x - xhat) ** 2))(
  frames, recon_frames)
model = update((cnn_enc, rnn_enc, rnn_dec, cnn_dec), grads)
```

Recurrent Autoencoders

Older language models used this approach

- Encode sequence of characters into meaning
- ELMo was a precursor to GPT

Applications and Objectives

Applications and Objectives

Can make many types of autoencoders

- Graph neural network autoencoders
- Transformer autoencoders

So far, we considered the reconstruction objective

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$$

This objective is useful for compression

Could we use autoencoders for other tasks?

Applications and Objectives

Task: Remove noise from an image



Can we do this with autoencoders?

Applications and Objectives

Reconstruction loss $\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$

Generate some noise $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma})$

Denoising loss $\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]} + \boldsymbol{\varepsilon}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$

Add noise to input $\boldsymbol{x} + \boldsymbol{\varepsilon}$, reconstruct $\boldsymbol{x}$ without noise

Autoencoder will learn to remove noise when reconstructing image

Applications and Objectives

Task: Remove blurring from an image



Applications and Objectives

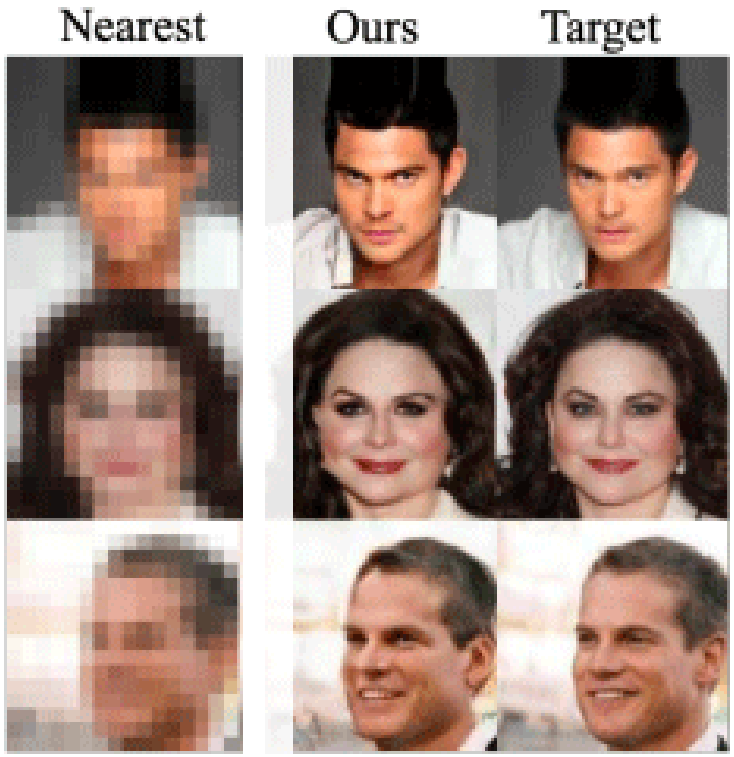
Reconstruction loss $\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$

Create blurry image $\text{blur}(\boldsymbol{x})$

Denoising deblur loss

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\text{blur}\left(\boldsymbol{x}_{[i]} \right), \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$$

Applications and Objectives



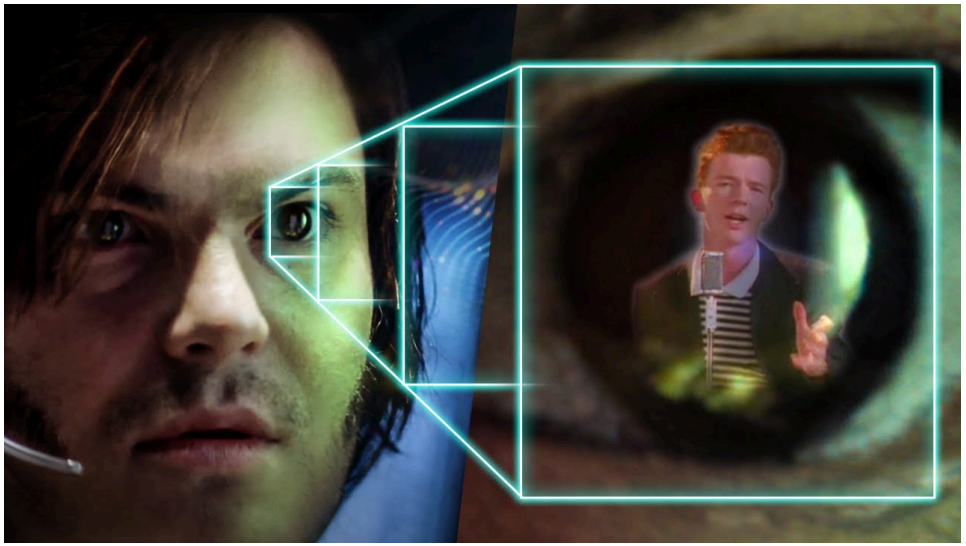
No more privacy in 2025

Applications and Objectives

Task: Fix blurry and noisy image

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\text{blur}\left(\boldsymbol{x}_{[i]} + \varepsilon \right), \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$$





Emergent Intelligence

Emergent Intelligence

Something very interesting happens with autoencoders

This becomes clear when we consider one last application

Emergent Intelligence

Task: Fix image with missing pixels

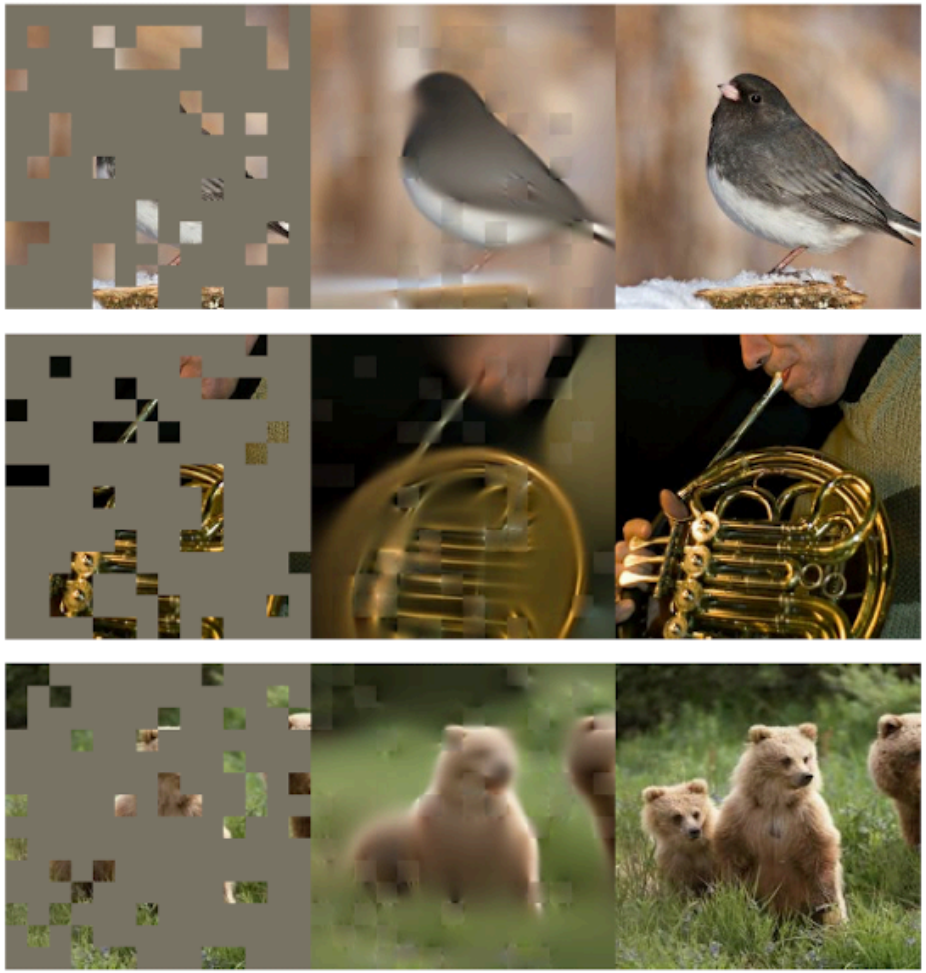
Reconstruction loss $\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$

Sample Bernoulli noise $\boldsymbol{b} \sim \mathcal{B}(0.2)$

Masked reconstruction loss

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]} \odot \boldsymbol{b}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$$

Emergent Intelligence



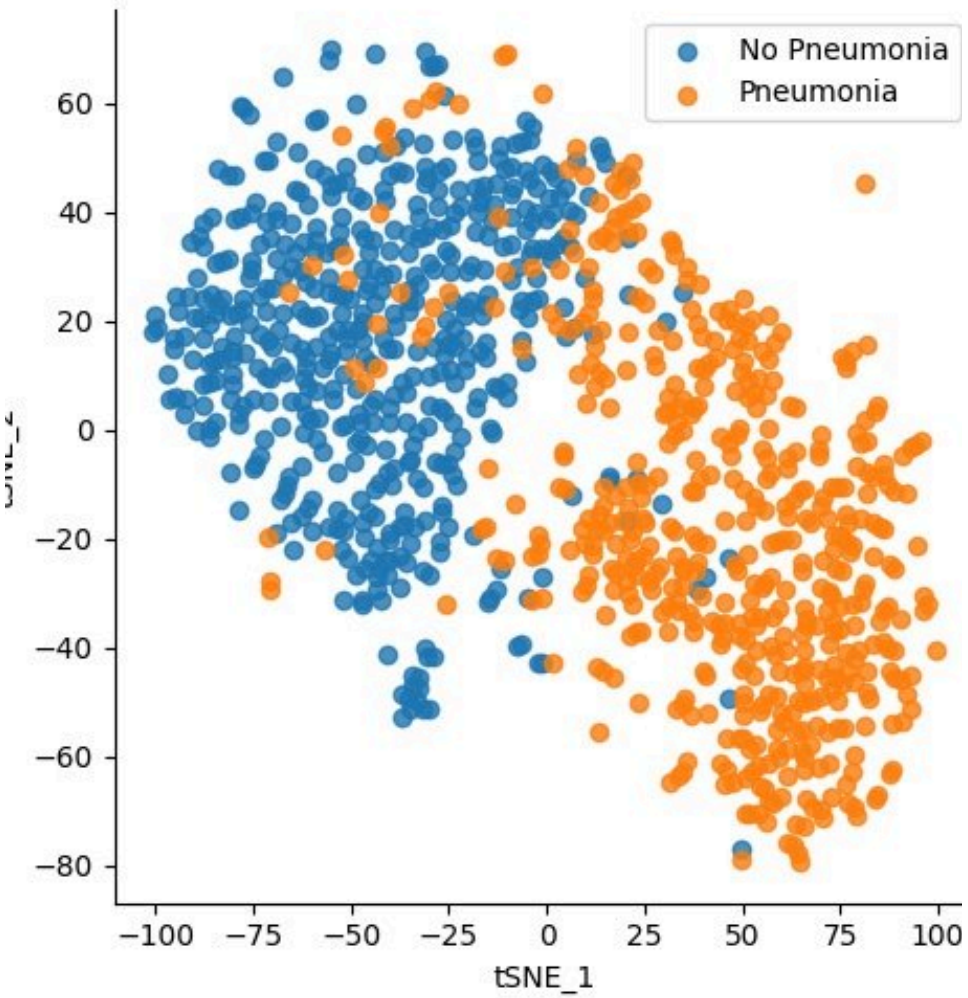
Understanding emerges from reconstruction objectives

Without labels, autoencoders learn what a bird is, what a bear is

They learn the structure and distribution of the input data X

If the input data X is from our world, then they begin to understand our world

Emergent Intelligence



X : Pictures of human lungs

$Z \in \mathbb{R}^2$

Learns the structure of lungs from images

Separates sick from healthy lungs without labels

- "Sick" or "healthy" information improves compression

Emergent Intelligence

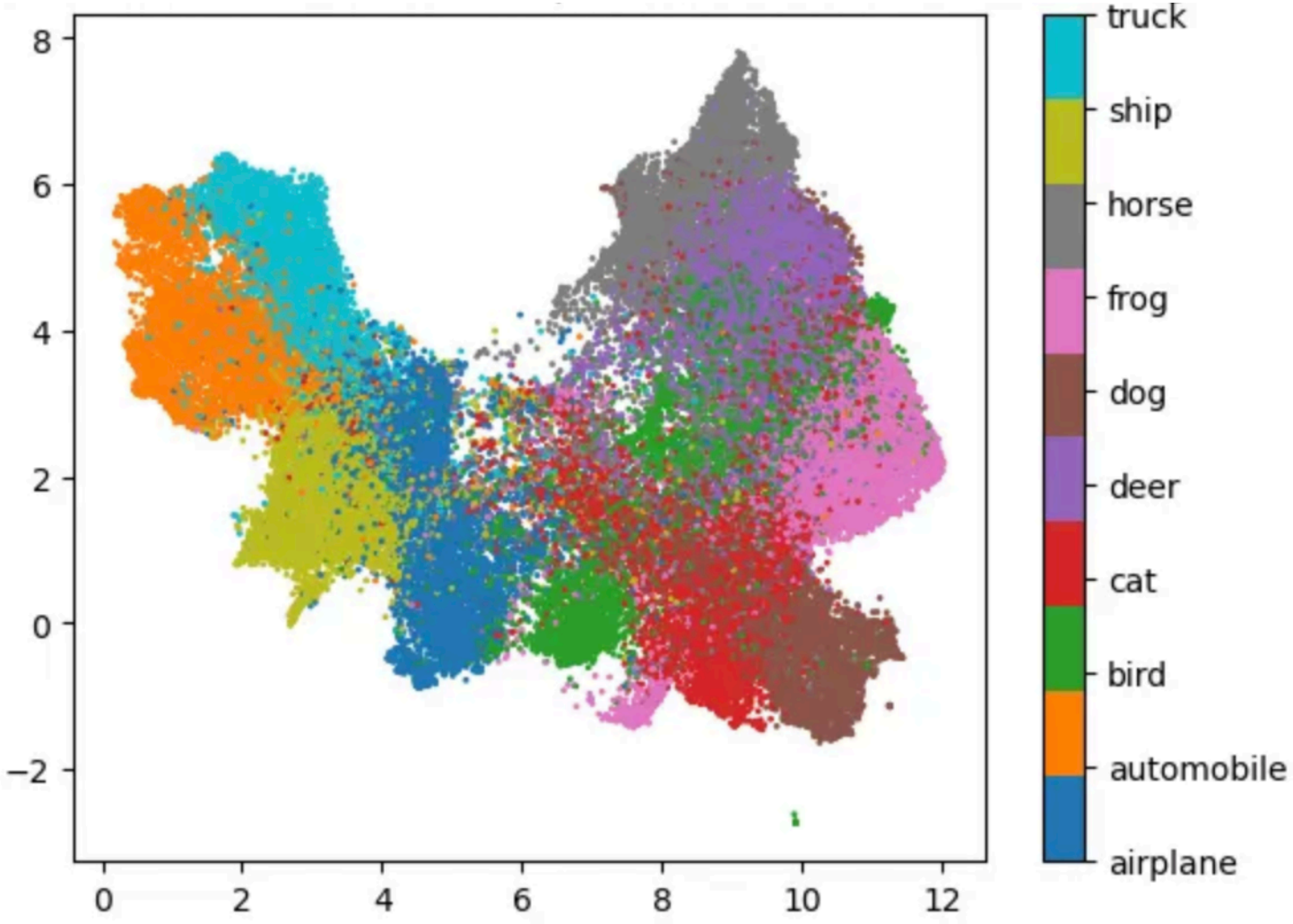
If the dataset is lung images, the model learns the structure of lungs

If the dataset is pictures from our world, then the autoencoders learn the structure of the world

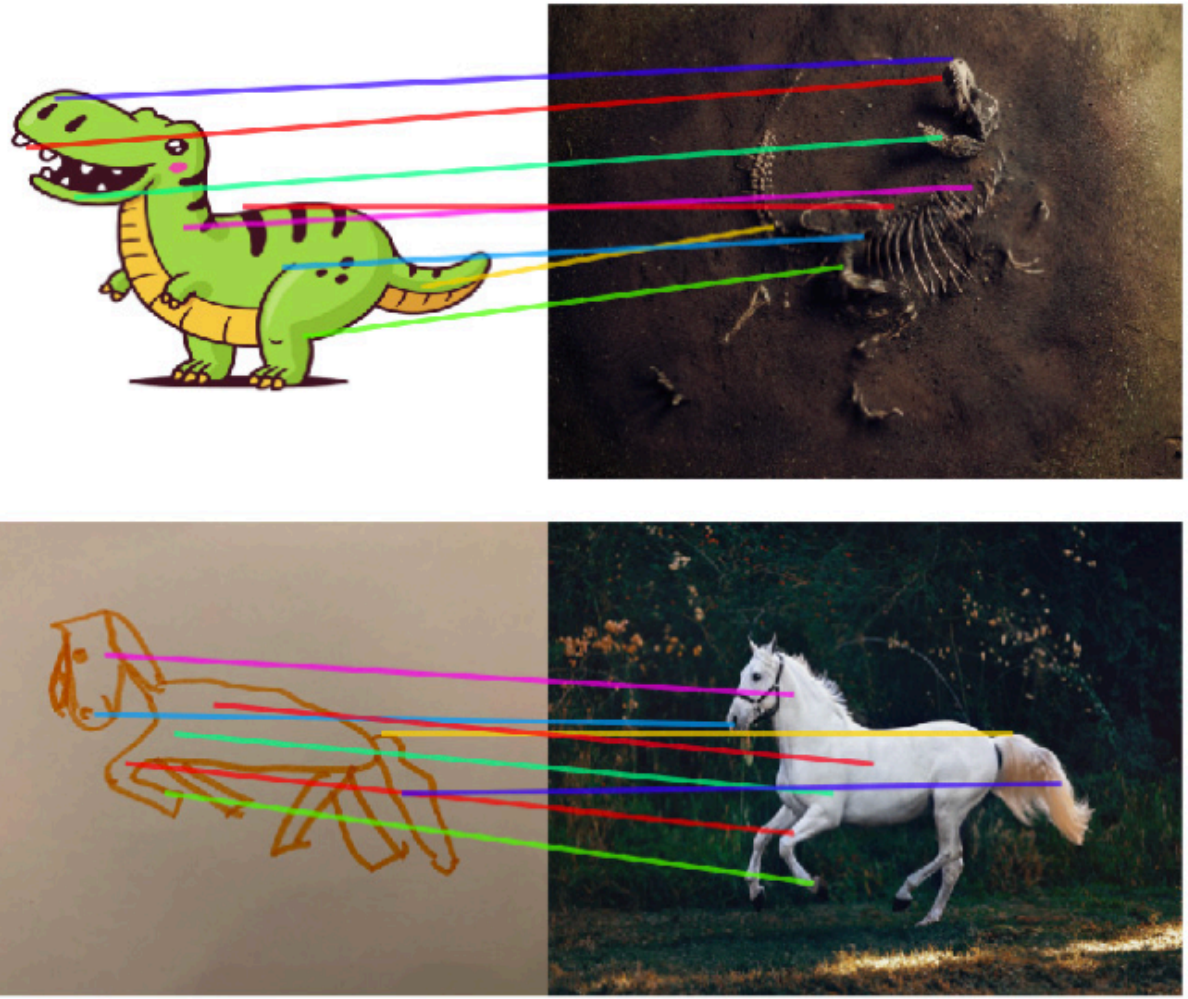
Nobody tells an autoencoder what a dog or cat is

They learn this on their own

Emergent Intelligence



Emergent Intelligence



Emergent Intelligence

Some say “neural networks do not understand”, they just learn patterns

Humans are also pattern recognition machines



Opinion: These networks understand our world as well as humans

Emergent Intelligence

Let us demonstrate this

https://colab.research.google.com/drive/18ZnYB79ZRag9HyiMJbxo9iT4lmIvM-fq?usp=sharing

https://www.youtube.com/watch?v=UZDiGooFs54